

CL-2006 Operating Systems Lab

Final Term Project

Note: Carefully read the following instructions as each carries a weightage.

- o **2-3 students** are allowed per group. No cross-section is allowed.
- o Your code should be clear and add exceptions where required (to avoid system crashing or accept incorrect data/options)
- o Project is to be submitted as a **zip file**.
- o Submit the whole project by **one member** of the group.
- o There must be a block of comments at the start of each function; the block should contain brief description about functionality of code.
- o Use understandable name of variables.
- o Proper indentation of code is essential.
- o Make a **report file on Microsoft Word** and paste all your **code** with all possible **screenshots** of every task (also the **information about modules** completed and **the sequence to run the program** and at the end submit **the video of the project**).
- o At the end when you are done with your tasks, make your submission on Google classroom. (Make sure your submission is completed).
- o Plagiarism will simply result in zero marks.
- o Please submit your file in this format **24F-1234_24F-1235_24F-1235_Project.zip**.

MiniOS X – A Multi-Process Operating System Simulator



Project Statement

An operating system (OS) is system software that manages computer hardware, software resources, and provides common services for computer programs and applications. In this project you will design and implement a Mini Operating System Simulator that demonstrates core OS concepts including process management, scheduling, synchronization, and resource allocation. The simulation should be made using and following all concepts studied in this course.

Key features the operating system should have:

- Your operating system should have a name of its own that will display on starting the program
Detail: You should make the program in such a way that when it starts it displays the name of your operating system on terminal using sleep command for some seconds like it is loading.
- The RAM hard drive and the number of cores will be given by the user while starting the OS

```
(base) Mateens-MacBook-Pro:desktop mateencheema$ g++ os.cpp -o OS  
(base) Mateens-MacBook-Pro:desktop mateencheema$ ./OS 2 256 8
```

This instance of OS will have 2GB RAM, 256GB Hard drive and 8 cores you can choose your own convention.

- The resources should be managed in your operating system RAM Hard drive and cores should be kept in mind while making new processes or threads.
- You will define basic applications and tasks that your operating system can perform:
 - Basic tasks would be like Notepad (writing into file with auto save function), Calculator, Time, creating a file, Move, Copy, Delete file, Check file info, Minigames (minesweeper etc.)
 - You should have minimum of 15 tasks in your operating system

Detail: Each task should have a defined task its own separate code file and it should make a new process; simple function calling is strictly not allowed the once a process is created it will send the creating process message that will contain memory required in Hard drive and RAM if available then the process will reply granted else the process should be terminated each process will have its own terminal so the output is not messed up.

This will help you to enable multi-tasking in your operating system.

- You can also manually create an interruption to stop the specific task or minimize it.

Detail: Each task should have a button to close the task in between so to make a simulation of interrupt.

- Your operating system should be able to multitask.
- Your operating system would have user mode and kernel mode (in kernel mode you can access hardware resources)

Detail: In kernel Mode user can close or delete the processes from memory meaning you can close running programs.

Project Working:

- Hardware Resources
 - Hardware resources must be provided for the OS to start.
- Operating System Boot
 - OS name appears on the boot screen.
 - After booting, basic tasks (like time and calendar) should auto-start.
 - You can add more startup tasks as needed.
- Instruction Guide
 - A guide should show all available tasks a user can perform.
- Task Execution
 - User selects a task → it starts in a new terminal using EXEC commands.
 - The task is given a specific location and size in RAM.
 - Task waits for its turn to execute, managed by scheduling techniques.
- Process Scheduling

- If the number of tasks increases, schedule them based on available CPU cores.
- A ready queue manages and sends tasks from RAM for execution.
- Multi-level Queue Scheduling
 - Different queues use different algorithms (e.g., FCFS, Round Robin)
 - Processes are assigned to queues based on type or priority
- Handling Interrupts
 - If an input or external interrupt occurs:
 - The task moves to Blocked State.
 - Processor can execute another task in the meantime.
 - After the interrupt ends, the original task resumes.
- Process Lifecycle
 - Each task executes based on its turnaround time (set during coding, not at runtime).
 - After a task finishes, it is removed from RAM.
- Data Storage
 - If the user saves something, it should be stored in the hard disk.

Operating system concepts:

Your project should involve the following OS concepts:

- Multitasking
- Context switching
- Resource allocation
- User mode and Kernel mode
- Process creation
- Threads
- EXEC commands

- Scheduling using mutual exclusion, semaphore and condition variable
- Scheduling techniques in ready queue. You should implement multilevel queue with different techniques on each level.

All the above concepts should be used in order to get full marks.

Project Phases & Deadlines

To ensure proper development, the project will be completed in the following phases:

- Phase 1: Project Proposal
- Phase 2: Design & Flowcharts
- Phase 3: Core Implementation
- Phase 4: Final Submission

Phase 1: Project Proposal

Submit a detailed project proposal in word document format containing:

- OS Name
- List of tasks to implement (minimum 12-15)
- Group members responsibilities
- Basic system idea

Phase 2: Design & Flowcharts

Submit the following artifacts:

- Flowcharts (process lifecycle, scheduling)
- System Architecture Diagram (MiniOS structure including processes, scheduler, memory, and interaction flow)
- Ready queue design
- Resource allocation strategy

Phase 3: Core Implementation

Submit the following modules implemented in your code:

- Process creation using fork/exec
- Basic multitasking

- Resource management working

Phase 4: Final Submission

Submit:

- Complete working project
- Report (with screenshots + explanation)
- Demonstration video

Note: Each phase submission is **mandatory**. Missing a phase may impact final evaluation.

Bonus Task: You will get **bonus marks** for implementing it using the graphics library.

Advanced Requirements (Mandatory Enhancements):

For your MiniOS functionality, the following features must also be implemented:

- Process Priority with Aging
 - Each process must have an assigned priority level.
 - If a process waits too long in the ready queue, its priority should gradually increase (aging technique) to prevent starvation.
- Basic Deadlock Detection
 - Your system should be able to detect a deadlock situation.
 - If circular waiting occurs between processes, display a message:
"Deadlock detected among processes"
- System Log File
 - Maintain a log file that records:
 - Process creation
 - Resource allocation
 - Process termination

```
[12:01] Process P1 created  
[12:02] RAM allocated: 200MB  
[12:03] Process terminated
```

Project Full Workflow:

Your program starts by requesting system resources and then allows the Operating System to load.

If you're using graphics or Windows Forms:

- Show icons for each available task just like real OS desktops.

If you're working in the terminal:

- Display a clear list of tasks that users can choose from.

Task Management Made Easy

- When a user selects a task:
 - They should have the option to Close or Minimize it.
 - Close: Ends the task, removes it from RAM, and frees up space.
 - Minimize: Keeps it in RAM, but hides it allowing the user to do something else.
- There should also be an option to switch to any currently running task in RAM perfect for multitasking!

Important Rule:

- The number of running tasks should not exceed available RAM size; manage this carefully.

Types of Tasks You Can Add

- Interactive Tasks (e.g., Calculator):

- Need full-time user input.
- Background Tasks:
 - Run quietly behind the scenes like music players or downloads.
- Processing-Only Tasks:
 - Do heavy lifting in the background without needing user input.

In this project, you'll not just write codes, but simulate how real operating systems handle multitasking, memory, and user interactions.

Types of Tasks:

- Full time response like a game. It can only stop by closing or minimizing it. They should occupy space according to their work.
- A background process like a song in background. You can implement it by using a beep sound playing in background for a specific time and it will stop according to its length according to a song. Other processes like this will be a copy file task or printing a file
- Some tasks that are performed on their own. Like a clock or saving a notepad file automatically

Each task should be able to close or minimize unless it is automatically performed like a clock. They can be changed from kernel mode. User can minimize a task anytime and can select from any other task to start and can add more tasks.

Each task will be added in ready queue and multiple tasks can run side by side. Like a game and songs. Like writing a notepad file and saving it. Like using calculator and printing a file.

More tasks should be added in the queue and run according to the synchronization.

User can shut down the operating system at any time. It will close all the tasks and close the program. Greeting message can be displayed on booting and shutting down too.

This project is not just about writing code. It is about understanding how an operating system thinks, decides, and manages the world inside a computer.

Good Luck!